Лабораторная работа №3
«ПРОЕКТИРОВАНИЕ РЕАЛИЗАЦИИ ФУНКЦИЙ СИСТЕМЫ С ПОМОЩЬЮ ДИАГРАММ ПОВЕДЕНИЯ»

3.1 Цель работы

Изучить методы динамического моделирования систем в языке UML, освоить построение диаграммы видов деятельности, обзорной диаграммы взаимодействия и диаграммы последовательностей.

Задание: уточнить архитектуру проектируемой системы путем моделирования поведения информационной системы в соответствии с выбранной предметной областью с помощью диаграммы видов деятельности, обзорной диаграммы взаимодействия и диаграммы последовательностей.

3.2 Теоретические сведения

Диаграмма видов деятельности

Диаграмма видов деятельности (activity diagram) отражает динамику системы и особенно полезна при описании поведения, включающего в себя большое количество параллельных процессов, а также для моделирования поведения системы в самом общем виде на этапе анализа (рис. 12).



Рис. 12. Узлы диаграммы видов деятельности

В данном представлении поведения системы упор делается на ее функциональные составляющие, или *действия* (Action). Действие представляет собой базовую единицу функциональности системы, не декомпозируемую в рамках содержащего их вида деятельности. Все действия являются предопределенными. Например, в языке UML определены действия для создания объектов, установки значений атрибутов, связывания объектов и т.д. Кроме того, имеются действия для выполнения работ, определяемых пользователем (в том числе и видов деятельности). Действия могут иметь входы и выходы, называемые *контактами* (Pin), которые связываются ребрами потоков объектов. Контакты – это разновидность узлов объектов, поэтому они временно сохраняют значения данных в потоке. В простейшем случае для начала выполнения действия требуется наличие всех его входных данных.

Вид деятельности (Activity) – это спецификация поведения системы в виде координируемой последовательности действий.

Подобно всем разновидностям работ в UML, вид деятельности можно инициировать с помощью *вызывающего действия* (CallAction). Вид деятельности содержит *параметры* (ActivityParameterNode) для получения

входных данных от вызывающего действия и предоставления ему выходных данных. Параметры не являются контактами, поскольку последние используются для соединения действий в потоке, а вид деятельности – это работа, вызываемая действием. Для доступа к значениям параметров из действий внутри вида деятельности параметры моделируются как особый вид узлов потоков объектов. Параметры размещаются на границе диаграммы, и ребра потока объектов соединяют их с контактами.

Раздел (ActivityPartition) представляет собой средство группировки действий в соответствии с некоторым признаком. Действие может входить одновременно в несколько разделов. Раздел может содержать несколько подразделов. Особым видом разделов являются так называемые внешние разделы, используемые для представления сущностей, внешних по отношению к системе. Внешние разделы помечаются стереотипом <<external>>. Наиболее часто разделы применяются на этапе определения требований для моделирования подразделений организационной структуры или отдельных исполнителей в бизнес-моделях. Так, в примере на рис. 13 разделы «Отдел заказов», «Бухгалтерия» и «Покупатель» отражают исполнителей бизнес-процесса обработки заказа.

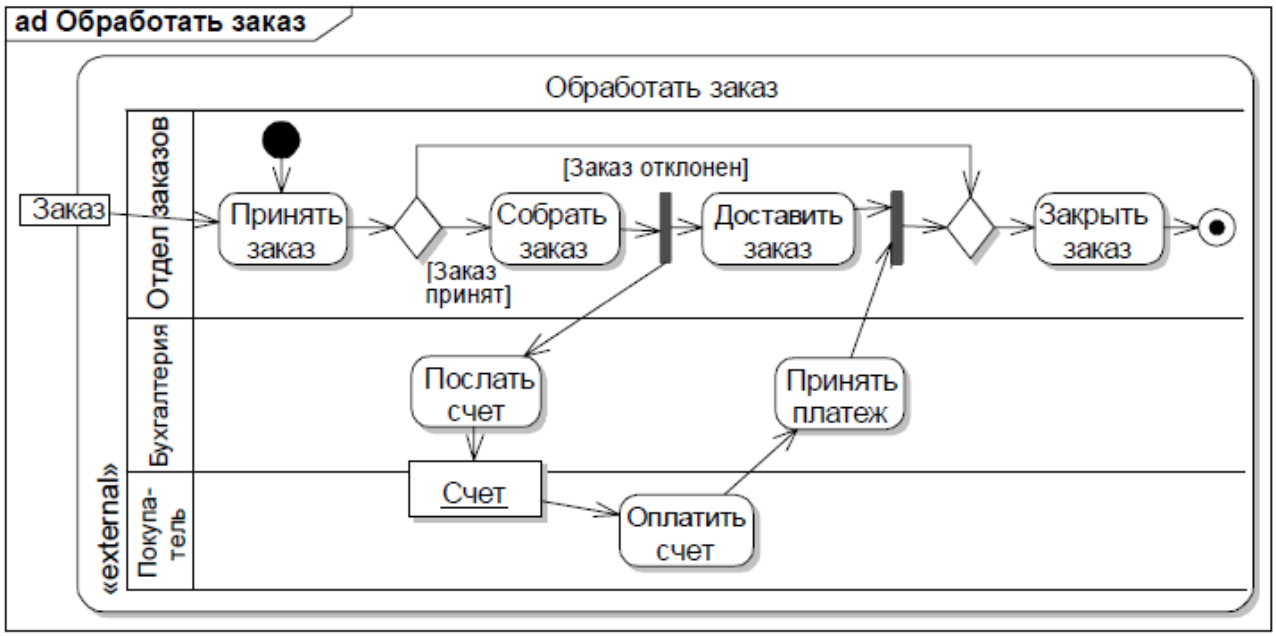


Рис. 13. Пример диаграммы видов деятельности

Диаграмма видов деятельности представляет собой граф, узлы которого соединены ребрами, представляющими потоки управления и данных. Маркеры управления и данных двигаются вдоль ребер и обрабатываются узлами, направляются на другие узлы или временно сохраняются. Каждые узел и ребро определяют, когда через них могут проходить управляющие воздействия и значения данных. Эти правила движения маркеров можно комбинировать с целью прогнозирования поведения всего графа.

В диаграммах видов деятельности имеется три вида узлов (условные обозначения приведены на рис. 12).

Узлы действия оперируют получаемыми управляющими воздействиями и значениями данных и предоставляют управление и данные другим действиям.

Узлы управления маршрутизируют перемещение маркеров управления и данных по графу. Эти узлы содержат конструкции для выбора между альтернативными потоками, для параллельного движения по нескольким потокам и т.д.

Объектные узлы временно удерживают маркеры данных, которые ожидают продолжения движения по графу.

Узлы связываются направленными ребрами двух типов:

– *ребра потоков управления* связывают действия. Они обозначают, что действие на конце ребра со стрелкой не может начаться до того, как закончится исходное действие. По ребрам потоков управления могут проходить только маркеры управления;
– *ребра потоков объектов* соединяют узлы объектов (в том числе, контакты действий и параметры вида деятельности). По ребрам потоков объектов могут проходить только маркеры данных.

Действие активизируется только при наличии маркеров управления на всех входных ребрах потоков управления и маркеров данных на всех входных контактах. Перед выполнением действия соответствующие маркеры

изымаются, а по его окончании на выходных ребрах потоков управления и выходных контактах размещаются маркеры управления и данных, после чего возможна активизация следующего действия.

Ветвление (DecisionNode) дает возможность показать разделение по условиям управляющих потоков. Данный узел имеет единственный вход и несколько выходов со сторожевыми условиями. Так как может выполниться только один из выходных переходов, то сторожевые условия должны взаимно исключать друг друга. Сторожевые условия изображаются в квадратных скобках около линии перехода. Если в качестве сторожевого условия используется [иначе], то это означает, что переход с данной меткой срабатывает в том случае, когда все другие переходы для данного ветвления являются ложными. *Слияние* (MergeNode) имеет несколько входов и единственный выход. Данный блок означает окончание условного поведения, которое было начато соответствующим ветвлением.

Параллельное поведение изображается при помощи *разделений* (ForkNode) и *объединений* (JoinNode). Последовательность выполнения параллельных действий является произвольной.

Конечный узел (ActivityFinalNode) завершает выполнение всех действий в виде деятельности (в том числе других видов деятельности, вызванных синхронно из текущего), удаляет все маркеры управления и данных, за исключением маркеров данных в выходных параметрах вида деятельности, и возвращает управление в вызвавшее его действие.

Конечный узел потока управления (FlowFinalNode) позволяет завершить отдельный поток управления, не затрагивая выполнение вида деятельности целиком. Это особенно полезно для моделирования видов деятельности, обрабатывающих потоки данных.

Диаграммы взаимодействия

Диаграммы взаимодействия (interaction diagrams) представляют собой модели, которые необходимы для описания поведения взаимодействующих групп объектов.

В языке UML существует четыре вида диаграмм взаимодействия: диаграмма последовательностей, диаграмма коммуникации, обзорная диаграмма взаимодействия и временная диаграмма (в пособии не рассматривается). Эти типы диаграмм дополняют друг друга, представляя взаимодействие элементов системы в динамике, но под различными углами зрения.

Диаграмма последовательностей

Диаграмма последовательностей (sequence diagram) отражает поток событий, происходящих при реализации некоторого прецедента, на этой диаграмме изображаются актеры, объекты, а также принимаемые и посылаемые ими сообщения (рис. 14).

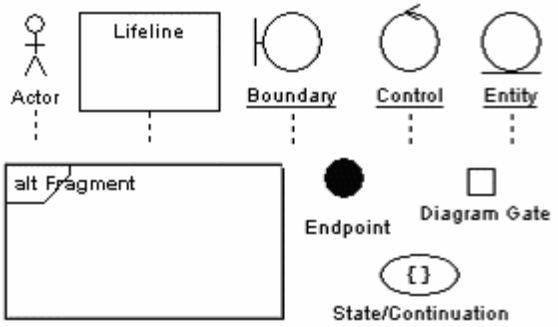


Рис. 14. Элементы диаграммы последовательностей

При построении диаграммы последовательностей в первую очередь отражается временная последовательность происходящих событий. На рис. 15 представлен пример диаграммы последовательностей, которая имеет два измерения: вертикальное направление представляет время, горизонтальное – различные объекты.

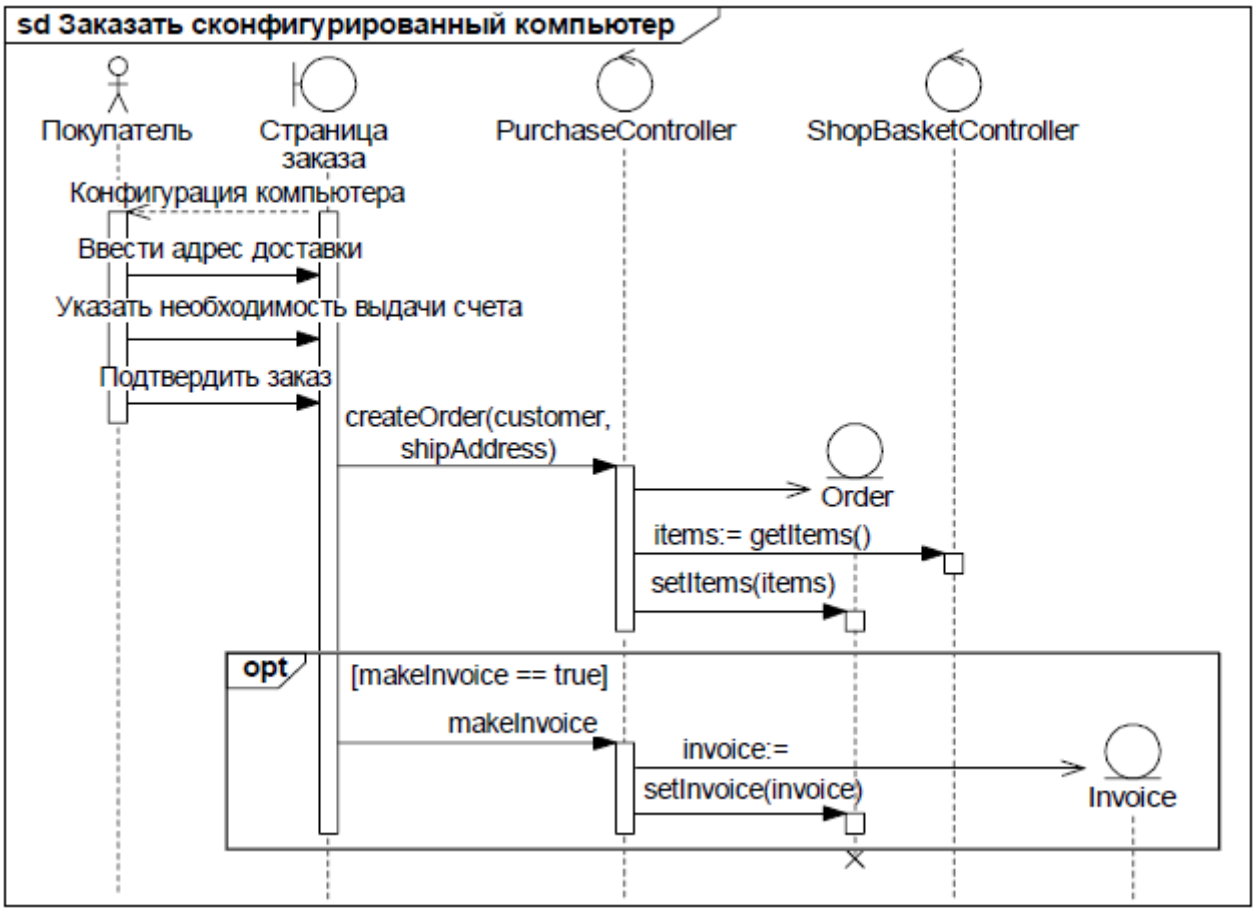


Рис. 15. Пример диаграммы последовательностей

Объект (Object) – это экземпляр класса, конкретная сущность или образец, он может инициировать некоторые события, которые могут влиять на систему. На диаграмме последовательностей все объекты расположены последовательно в верхней ее части, за исключением объектов, создаваемых в результате тех или иных сообщений (примерами последних являются объекты Order и Invoice на рис. 15).

От каждого объекта вниз отходит штриховая линия, называемая *линией жизни* (Lifeline) объекта. На ней показывают все, что происходит с объектом с момента его создания и до разрушения.

Частным случаем объекта может быть актер, он используется в том случае, когда необходимо показать связь участников и прецедентов.

Сообщения, передающиеся от одного объекта к другому, представляются стрелками между линиями жизни этих объектов. Порядок следования сообщений устанавливается сверху вниз. Каждое сообщение

имеет как минимум *имя сообщения*, а также может иметь аргументы и некоторую управляющую информацию.

Для того чтобы показать момент времени, в который объект является активным, используются *прямоугольники активности,* их изображают поверх линии жизни.

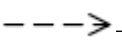
Управляющая информация может быть представлена в виде условия, которое указывает, когда сообщение может быть передано, либо при помощи *маркера итерации*, показывающего, что сообщение посылается множество раз. Такая итерация дается в следующем виде: *[условие итерации].

В зависимости от типа сообщение изображается при помощи различных линий:

 – вызов процедуры или другого потока управления;

 – поток управления, показывает направление потока управления и последовательности шагов;

 – асинхронное стимулирующее воздействие;

 – возврат из процедуры, используется для того, чтобы показать, что каждая процедура возвращает управление после своего завершения. Обычно стрелка возврата указывается только в том случае, если это вносит в диаграмму дополнительную ясность.

В качестве элементов диаграммы последовательностей могут применяться так называемые *фрагменты взаимодействия* (InteractionFragment), которые по своим свойствам не отличаются от полного взаимодействия объектов и позволяют наглядным образом группировать последовательности сообщений.

Включение (InteractionUse) представляет собой ссылку на имеющееся в модели взаимодействие вместо копирования последнего на диаграмму, что поддерживает декомпозицию и повторное использование отдельных взаимодействий. Включение обозначается как фрагмент взаимодействия с

оператором «ref», содержимое которого состоит из названия соответствующего взаимодействия.

Комбинированный фрагмент (CombinedFragment) представляет собой выражение над фрагментами взаимодействия, состоящее из оператора и операндов – фрагментов взаимодействия. С каждым операндом может быть связано сторожевое условие. В языке UML над фрагментами взаимодействий определены следующие основные операторы:

- alt – выполняется не более одного операнда, для которого сторожевое условие истинно;
- opt – необходимость выполнения единственного операнда определяется его сторожевым условием;
- break – единственный операнд выполняется вместо всех остальных сообщений в объемлющем фрагменте взаимодействия в случае, если его сторожевое условие истинно;
- loop – циклическое выполнение единственного операнда, минимальное и максимальное количество итераций указывается в операнде;
- par – параллельное выполнение операндов при сохранении последовательности сообщений в каждом из них;
- strict – строго последовательное выполнение операндов;
- seq – условно последовательное выполнение операндов, при котором упорядочиваются только сообщения, относящиеся к одной и той же линии жизни; в последнем случае сообщение из первого операнда выполняется прежде сообщения из второго операнда и т.д.;
- critical – критический участок, операнды выполняются без перекрытия во времени с любыми другими сообщениями, относящимися к тем же линиям жизни, которые задействованы в операндах.

На рис. 13 приведен пример условного комбинированного фрагмента opt, включающего единственный операнд, для которого определено необходимое сторожевое условие.

Инвариант состояния (StateInvariant) представляет собой ограничение на состояние объекта в процессе выполнения взаимодействия. Если данное ограничение не выполняется, то вся предыдущая последовательность сообщений считается неверной. Пример инварианта состояния приведен на рис. 16, где с его помощью показано, что возможна отмена только тех заказов, которые еще не доставлены.

Диаграммы последовательностей являются наглядным и легко читаемым средством описания функционирования системы. Они помогают быстрее разобраться в процессах поведения системы.

Диаграмма коммуникации

Диаграмма коммуникации (communication diagram) отображает ту же информацию, что и диаграмма последовательностей, но на диаграмме коммуникации зависимость от времени указывается посредством нумерации сообщений. На диаграмме коммуникации отражается распределение процессов между объектами и их зависимости друг от друга, что очень полезно при разработке различных проектов. Основной целью построения данной диаграммы является понимание структурной организации занятых в системе объектов, принимающих и передающих сообщения. На рис. 16 показан пример диаграммы коммуникации.

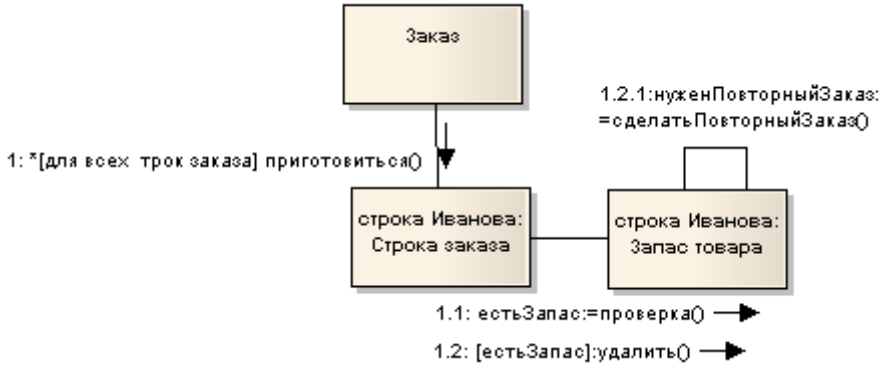


Рис. 16. Пример диаграммы коммуникации

Обзорная диаграмма взаимодействия

Обзорная диаграмма взаимодействия (interaction overview diagram) отражает потоки управления, возникающие при реализации некоторого прецедента. Она является частным случаем диаграммы видов деятельности и может содержать те же элементы, с тем исключением, что вместо узлов действий и объектов используются диаграммы взаимодействия или ссылки на них (условные обозначения приведены на рис. 17, пример – на рис. 18).

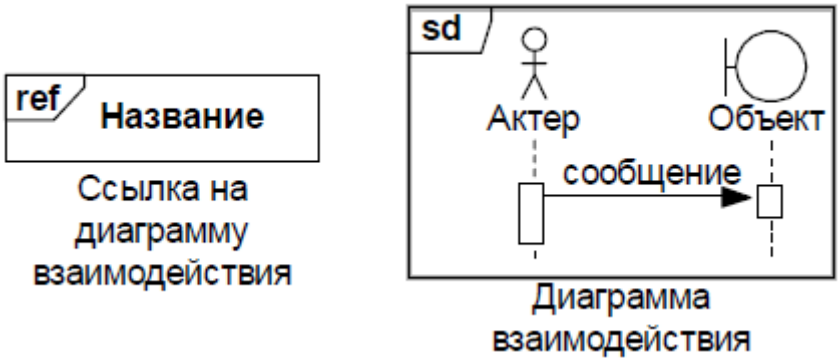


Рис. 17. Элементы обзорной диаграммы взаимодействия

Обзорная диаграмма взаимодействия позволяет альтернативным способом представить следующие виды комбинированных фрагментов взаимодействия: фрагменты alt и opt заменяются узлом разветвления и парным ему узлом слияния; фрагмент par заменяется узлом разделения и парным ему узлом объединения; фрагмент loop представляется в виде простого цикла.

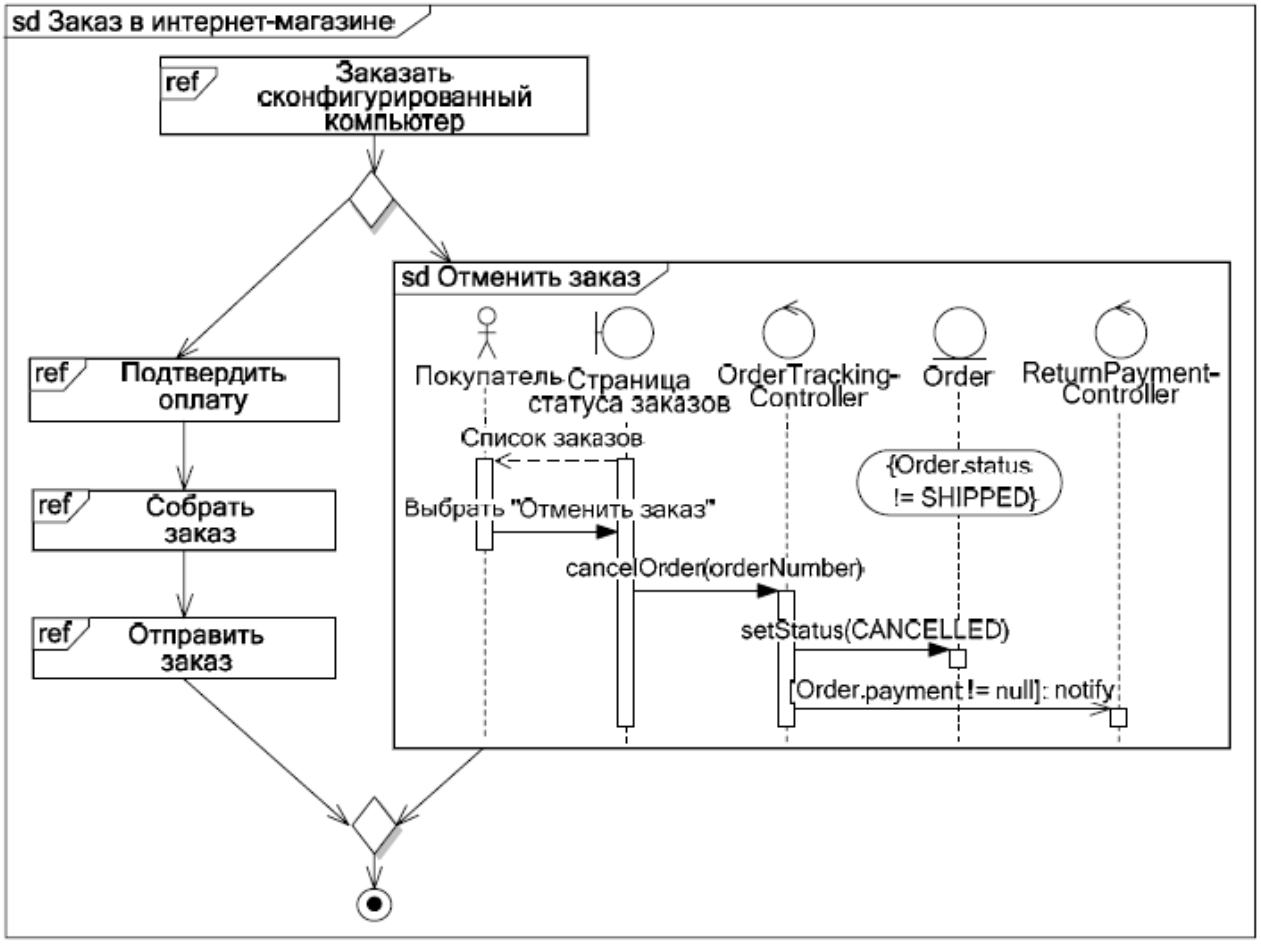


Рис. 18. Пример обзорной диаграммы взаимодействия

В силу этого в рассматриваемых диаграммах условные и параллельные потоки управления должны быть строго вложенными.

Данный вид диаграмм соединяет особенности диаграмм видов деятельности и диаграмм взаимодействия, акцентируя внимание разработчика не только на прохождении потоков управления, но и на реализации отдельных действий в виде взаимодействия объектов.

3.3 Порядок выполнения

1. Изучить теоретические сведения по разработке диаграмм поведения в языке UML.

2. В пакете Enterprise Architect при создании новой диаграммы необходимо выбирать группу *UML Structural* для выбора диаграмм структуры, и *UML Behavioral* для выбора диаграмм поведения.

3. Построить диаграммы видов деятельности (не менее двух) для прецедентов разрабатываемой системы, в которых должны быть использованы ветвления и параллельные потоки управления.

4. Построить диаграммы последовательностей (не менее двух) для отдельных действий одной из диаграмм видов деятельности. В данных диаграммах необходимо использовать не менее двух комбинированных фрагментов с различными операторами.

5. Построить диаграммы коммуникации.

6. Построить обзорную диаграмму взаимодействия на основе диаграммы видов деятельности, разработанной в п. 2. Обзорная диаграмма взаимодействия должна содержать ссылки на построенные диаграммы последовательностей, а остальные действия должны быть представлены фрагментами взаимодействия непосредственно на диаграмме.

7. Кратко описать построенные диаграммы.

3.4 Содержание отчета

1. Цель работы.

2. Краткое описание разрабатываемой системы (в соответствии с выданным индивидуальным заданием).

3. Построенные диаграммы поведения с краткими пояснениями.

4. Вывод.

3.5 Контрольные вопросы

1. Перечислите виды диаграмм поведения в языке UML, поясните их основные отличия.

2. Дайте определение действия и деятельности, назовите их основные отличия.

3. Назовите и приведите условные обозначения основных элементов диаграмм последовательностей.

4. Поясните связь диаграмм поведения с моделью прецедентов.